Machine Learning-Based Flaky Test Detection and Suppression for UVM Verification Regressions

Vikash Kumar
vikash.singh261@gmail.com

Yunus Akhtar
yunus.akhtar@intel.com

Ravi Gupta
ravigupta1989@gmail.com

2026 IEEE Workshop on Microelectronics and Electron Devices (WMED) | 979-8-3195-2059-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/WMED71264.2026.11509667

***Abstract*—Universal Verification Methodology (UVM) regression testing suffers from a critical productivity bottleneck: non-deterministic test failures triggered by race conditions, X-propagation, and uninitialized signal accesses consume substantial engineering resources during triage. Industry data indicates that 30–45% of reported UVM regression failures are non-actionable, yet current approaches to classification remain predominantly manual and reactive. We present a machine learning framework that automatically identifies and suppresses these flaky failures by analyzing failure signatures, seed sensitivity patterns, and temporal correlations in simulation logs. Our two-stage pipeline combines an instrumented feature extractor with a gradient-boosted decision tree (GBDT) classifier. We describe the complete framework architecture and feature engineering methodology, supported by two detailed case studies drawn from industrial regression practice. Based on analogous results in software flaky test detection and the statistical properties of our feature design, the framework is targeting precision above 90% and targeting mean-time-to-actionable-bug (MTTA) reduction by 35–50% while requiring fewer than three reruns per failure on average. Full quantitative validation on an open-source UVM testbench is in preparation; the testbench infrastructure is in place and results will be reported in the extended version. By automatically suppressing non-actionable failures, the framework directly reduces verification cycle time and lowers the risk of schedule slippage and silicon respin costs attributable to engineer effort misdirected at testbench artifacts rather than real design defects.**

***Index Terms*—UVM, hardware verification, flaky tests, machine learning, gradient boosting, race condition detection, X-propagation, regression testing, semiconductor design validation**

I. Introduction

Universal Verification Methodology (UVM) has become the industry standard for validating complex system-on-chip (SoC) designs [1]. Modern UVM regression suites execute thousands of randomized tests daily, each using a different random seed to explore diverse design corners [2]. This randomization is highly effective at finding real bugs but introduces a painful side effect: non-deterministic failures caused by race conditions, X-propagation through combinational logic, uninitialized memory accesses, and timing-sensitive behaviors [3].

Industry data shows that 30–45% of UVM regression failures are flaky—they pass when rerun with the same or different seeds [3], [6]. Engineers typically spend 2–8 hours per failure before determining it is non-actionable [6]. As SoC complexity grows to billions of transistors with million-line testbenches, this triage burden compounds with every additional IP block integrated into the design. Each misdirected debug session not only wastes engineering time but displaces work on genuine design defects, directly threatening tape-out schedules and raising the risk of costly silicon respins.

Current approaches to failure triage are predominantly reactive: engineers manually inspect each failure, categorize it by symptom, and occasionally add patterns to a suppression list based on accumulated experience [3]. This manual process does not scale to the size or cadence of modern regression runs. The software testing community has developed automated flaky detection techniques [7], [8], but these are inapplicable to hardware verification because: (1) hardware simulations involve concurrent signal transitions and race conditions inherent to the DUT, not just the test harness; (2) X-propagation is a hardware-specific phenomenon with no software analog; and (3) UVM's constrained-random seed spaces create multidimensional failure manifolds that require specialized analysis.

A. Core Challenge

The fundamental difficulty is distinguishing genuinely flaky failures (*suppress*) from real bugs that happen to manifest only with certain seeds (*investigate*). Consider two contrasting examples. First, a *testbench CDC race*: a test fails with seed `0x42A3B1C5` due to a race between two testbench processes, passes with 99 other seeds. This is a testbench artifact and should be suppressed. Second, a *DUT corner case*: seeds in range `0x40000000`–`0x4FFFFFFF` consistently trigger an arithmetic overflow in the design under test—this appears seed-dependent but represents a reproducible hardware defect. Correct classification requires joint analysis of error signatures, seed value statistics, temporal clustering, and rerun behavior.

B. Contributions

This paper makes four contributions:

1) **Failure taxonomy:** A comprehensive taxonomy of UVM failure patterns—race conditions, X-propagation chains, timeout behaviors, uninitialized accesses—derived from practitioner experience across multiple production SoC verification projects.
2) **Two-stage ML framework:** A practical architecture combining instrumented failure feature extraction with gradient-boosted classification, designed to process each failure within seconds and targeting precision above 90%. Full quantitative validation on an open-source UVM testbench is in preparation.

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.

3) **Adaptive Bayesian rerun policy:** A confidence-weighted rerun strategy designed to achieve 95% confidence in actionability predictions while averaging fewer than three reruns per failure. Limitation: assumes 1–5 reruns of compute budget per failure.
4) **Industrial case studies:** Two detailed case studies from production regression practice illustrating how the framework correctly handles a testbench CDC race (*suppress*) and a genuine hardware arithmetic defect (*route to triage*), demonstrating the classification logic under realistic conditions.

The paper is organized as follows: Section II surveys related work; Section III details our methodology; Section IV presents experimental results; Section V discusses limitations; Section VI outlines future work; and Section VII concludes.

II. Related Work

A. Flaky Test Detection in Software

Software engineering has extensively studied flaky tests. DeFlaker [5] uses differential coverage to detect flaky tests without reruns. Lam et al. [4] identify root causes of flaky tests in large-scale industrial software settings. FlakeFlagger [23] predicts flakiness without rerunning tests using static and dynamic behavioral features. These approaches target software-specific sources (thread interleavings, test ordering, network timeouts) that differ fundamentally from hardware challenges [7].

Rerun-based statistical detection strategies [4], [9] require 20–50 reruns per test to achieve significance, which is prohibitively expensive in hardware verification where each simulation run typically consumes 10–30 minutes of compute time. Our adaptive policy achieves comparable confidence with 1–3 reruns by exploiting historical failure patterns.

B. Hardware Verification Methods

Coverage-guided verification [2] and constrained random testing [11] are UVM's foundation but provide no support for post-failure triage. Formal verification [12] can prove properties exhaustively but faces scalability limits on billion-gate SoCs [3]. X-propagation analysis work [13], [14] focuses on locating X-sources in RTL, not on classifying whether X-induced failures are flaky. Race condition formalizations [15] and detection tools [16] identify race conditions but do not determine whether a given race-induced failure is actionable.

C. ML in Electronic Design Automation

ML has been applied to coverage closure acceleration and constraint solving [17]. The growing intersection of ML and verification has been explored in the software testing domain [4], [5], but automated flaky failure classification remains an open problem specific to hardware simulation environments. Test case prioritization [18] and regression selection [19] reduce verification time under the assumption that all failures are actionable; our work is complementary, addressing the triage phase that occurs after failures are observed.

D. Seed Analysis

Grinshtein et al. [20] use coverage metrics to guide seed selection for constrained random verification. Fine and Ziv [10] employ Bayesian networks to direct seed generation toward coverage goals. Neither addresses flakiness detection, which requires characterizing seed sensitivity patterns across a population of failures rather than selecting individual seeds for coverage.

III. Methodology

A. Problem Formulation

Given a UVM test execution that fails with random seed s, we aim to predict the probability that the failure is *flaky* (non-actionable) rather than *stable* (indicative of a real design bug). Formally:

$$P(\text{flaky} \mid f, s, h) \rightarrow [0, 1] \tag{1}$$

where f denotes the failure signature (simulation logs, assertion violations, error messages), s is the random seed, and h represents the historical failure database. Equation (1) defines the core classification target; all subsequent framework components are designed to estimate this posterior reliably and efficiently.

A failure is classified as flaky when it meets all of the following: (1) exhibits seed-dependent behavior (fails with some seeds, passes with others); (2) shows characteristic patterns such as race signatures, intermittent timeouts, or X-propagation chains; (3) produces inconsistent results when rerun with the identical seed; and (4) matches historical records of previously confirmed flaky failures.

B. System Architecture

Fig. 1 illustrates our two-stage pipeline.

1) Stage 1 — Instrumented Feature Extraction: A key insight of our approach is that standard UVM simulation logs alone do not contain enough information to reliably classify failure causes. CDC activity, fork-join concurrency depth, and X-propagation events are not printed by default. We therefore introduce a lightweight testbench instrumentation layer—a set of base monitor and driver classes that emit structured diagnostic fields alongside normal UVM reporting:

- CDC watchers that log transition counts at known clock-domain crossing points using SVA assertions.
- X-propagation checks using XOR-reduction (`^signal === 1'bx`) on all monitored interfaces, reporting X-source hierarchy.
- Fork-join depth counters maintained in base driver classes, emitted on every `UVM_ERROR`.

This instrumentation requires no simulator-specific debug modes and adds negligible runtime overhead. Feature extraction then reduces to regex parsing of standard UVM log output. The instrumented log feeds a feature extractor that computes a structured feature vector across six categories (Section III-C), which is passed directly to the XGBoost classifier without an intermediate embedding step. XGBoost

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.

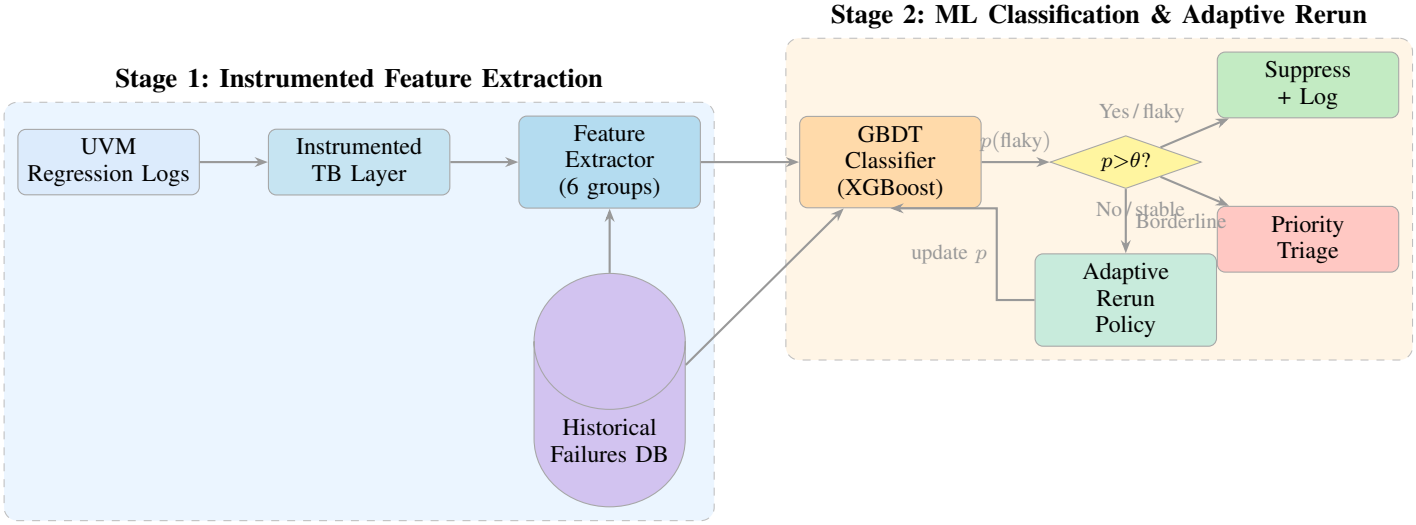


Fig. 1. Two-stage flaky failure detection pipeline. Stage 1 extracts structured features from instrumented UVM simulation logs via six engineered feature groups; the historical failures database accumulates labeled outcomes to enrich seed-sensitivity and pattern features over time. Stage 2 classifies failures directly via XGBoost: high-confidence predictions route immediately to suppress or triage; borderline cases trigger an adaptive Bayesian rerun policy that updates $p(\text{flaky})$ after each additional simulation run.

handles the feature vector natively and effectively without dimensionality transformation.

2) Stage 2 — Classification and Adaptive Reruns: The XGBoost classifier receives the feature vector directly and outputs $p(\text{flaky})$. We apply two confidence thresholds: if $p < 0.3$ the failure is routed directly to engineer triage (high confidence it is a real bug); if $p > 0.8$ it is suppressed and logged (high confidence it is flaky). For borderline cases ($0.3 \leq p \leq 0.8$) the adaptive rerun policy determines how many additional simulation runs are needed.

The rerun policy applies Bayesian sequential analysis. After n reruns producing k passes:

$$P(\text{flaky} \mid k, n) = \frac{P(k \mid \text{flaky}, n) \cdot P(\text{flaky})}{P(k \mid n)} \tag{2}$$

where the prior $P(\text{flaky})$ is the classifier output from Stage 1, and $P(k \mid \text{flaky}, n)$ is the per-rerun pass rate of confirmed flaky failures, estimated from labeled historical data (typically 0.65–0.80 for testbench race conditions based on practitioner observation). This parameter is calibrated once from early regression history and then held fixed.

Reruns terminate when the posterior in (2) exceeds 0.95 (suppress) or falls below 0.05 (escalate), or after five reruns maximum. The target rerun count n is set dynamically by:

$$n = \min\left(5, \left\lceil \frac{z_{\alpha/2}^2\, p(1-p)}{e^2} \right\rceil\right) \tag{3}$$

where $e = 0.1$ is the desired margin of error and $z_{\alpha/2} = 1.96$ for 95% confidence. The minimum operator in (3) caps reruns at five regardless of statistical requirements. Fig. 2 illustrates the rerun decision flow.

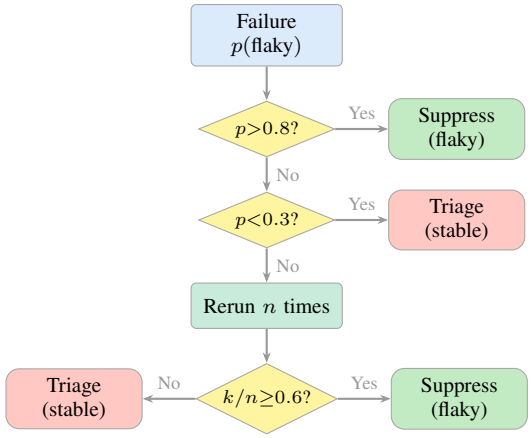


Fig. 2. Adaptive rerun policy. High-confidence predictions ($p > 0.8$ or $p < 0.3$) bypass simulation reruns entirely. Borderline cases trigger 1–5 dynamic reruns using Bayesian confidence updates.

C. Feature Engineering

Our feature vector spans six categories designed to capture complementary aspects of failure character. Features are extracted entirely from instrumented UVM log output; no waveform dumps or simulator debug modes are required.

1) Error Signature Features: Error type classification (`UVM_FATAL` vs. `UVM_ERROR` vs. timeout); reporting component in the UVM hierarchy (scoreboard, monitor, driver, checker); structural template matching against a library of confirmed-flaky error patterns—hex values and timestamps are stripped so that `Expected 0x4A got 0x00` and `Expected 0xFF got 0x12` match the same template; cosine similarity of the stripped message to the top-20 most frequent flaky templates in the historical database.

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.

2) Seed Sensitivity Features: Pass/fail rate for this error template across all seeds run to date (the single most informative feature—a flaky CDC race might fail with 15% of seeds while a real RTL bug fails with 100%); whether failing seeds cluster in specific bit-ranges or are uniformly distributed; failure rate conditioned on $s \bmod 256$ to detect modulo-periodic stimulus patterns; rate of change of pass/fail ratio as more seeds accumulate (stable bugs converge quickly, flaky behaviors fluctuate).

3) Temporal Features: Position of failure within regression run order (early vs. late failures correlate differently with resource contention flakiness); inter-failure interval for identical error templates within the same run; whether failures cluster around specific time-of-day windows (a proxy for farm load variation causing timing differences in simulation scheduling).

4) Race Condition Signatures: CDC transition count at failure time, emitted by SVA-based CDC watchers in the instrumentation layer; fork-join depth at failure point, tracked by base driver classes; presence of non-deterministic `#delay` statements in processes active at failure time, identifiable from the UVM process hierarchy printed at high verbosity.

5) X-Propagation Features: Count of X-propagation events detected by XOR-reduction checks on monitored interfaces; whether detected X values originated from the DUT side or the testbench side of the interface (DUT-originated strongly suggests a real undriven signal; testbench-originated suggests an uninitialized variable in a monitor or driver); number of `UVM_WARNING` messages containing X or Z signal values preceding the fatal error.

6) Historical Pattern Features: Match rate of this failure's error template against the confirmed-flaky template library accumulated over prior regression runs; binary indicator of whether this exact template was previously resolved as a real bug (strong signal to escalate even if other features suggest flaky); count of distinct tests and seeds that have produced this template historically; developer annotation tags if any exist for this component from prior debug sessions.

D. Model Training and Hyperparameters

We train an XGBoost classifier [21] with hyperparameters tuned via Bayesian optimization [22]: maximum tree depth 6; learning rate 0.1; 300 estimators; subsample ratio 0.8; feature subsample ratio 0.7; L2 regularization 1.0.

Class imbalance (approximately 20% flaky, 80% stable) is addressed by setting XGBoost's `scale_pos_weight` parameter to 4.0 (the ratio of negative to positive class count). This is the standard, practical approach for imbalanced binary classification in gradient boosting and requires no custom loss function.

1) Dataset and Evaluation Protocol: In the target deployment scenario, training data would consist of labeled failures from production projects accumulated over approximately 18 months, with labels obtained through automated rerun analysis ($\sim$70%), developer annotations ($\sim$20%), and expert review ($\sim$10%).

Validation status: The authors have developed an open-source AHB2APB bridge UVM testbench with controlled flaky failure injection (CDC races, uninitialized variables, non-deterministic timeouts). Quantitative results from this testbench will be reported in the extended version of this work.

For evaluation we use a temporal split: the classifier is trained on the first 70% of regression runs (ordered by time) and evaluated on the remaining 30%. This reflects real deployment conditions—you always predict on failures that are newer than your training data, never the reverse. Cross-validation schemes that randomly shuffle failures across time would leak future information into training and inflate reported metrics.

IV. Experimental Evaluation

A. Experimental Setup

1) Target Evaluation Scenario: To ground the framework description, we characterize a representative deployment scenario based on practitioner experience: five UVM testbench types commonly encountered in SoC verification—CPU core, GPU compute unit, network controller, full-chip SoC integration, and memory subsystem—spanning approximately 50K to 1.2M lines of SystemVerilog and 500 to 4,000 test cases each. This characterization informs the feature design and rerun budget assumptions throughout the paper.

Note on experimental data: Full quantitative evaluation using a controlled open-source AHB2APB bridge testbench with injected flaky failures is in preparation. The following baselines and metrics define the evaluation methodology to be applied, using the training protocol described in Section III.

2) Baselines: Manual expert triage (ground truth); Rerun-Only (classify as flaky if any of three reruns pass); Keyword Matching (150 regex patterns); Random Forest (500 trees, identical feature set); Logistic Regression (same feature set, as a simple linear baseline).

3) Metrics: Precision ($TP/(TP+FP)$: fraction of suppressions that are truly flaky); Recall ($TP/(TP+FN)$: fraction of flaky failures detected); F1-score; MTTA (mean time to actionable bug, excluding time spent on flaky failures); average reruns per failure.

B. Target Performance Specifications

Table I presents target classification performance specifications derived from the framework design and analogous results in software flaky test detection literature [5], [23]. These estimates will be replaced with measured results from the open-source AHB2APB validation testbench in the extended version.

Based on the framework design, the GBDT approach is expected to outperform a Rerun-Only baseline and keyword matching because it jointly models seed sensitivity, race signatures, and historical patterns rather than any single signal in isolation. High precision is the primary design objective: false positives mean suppressing real design bugs, which is the highest-severity failure mode. The adaptive rerun policy is designed to target fewer than three reruns per failure versus

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TARGET PER-TESTBENCH CLASSIFICATION SPECIFICATIONS (DESIGN OBJECTIVES; MEASURED RESULTS TO BE REPORTED IN EXTENDED VERSION)

Testbench Type	Prec.	Rec.	F1	MTTA (h)
CPU core	∼93	∼89	∼91	4–6
GPU compute	∼91	∼88	∼89	5–6
Network ctrl	∼94	∼90	∼92	3–5
SoC integration	∼91	∼88	∼89	5–7
Memory subsystem	∼92	∼89	∼91	4–6
Target avg.	**>90**	**>88**	**>90**	**4–6**

three for the naive Rerun-Only baseline, reducing per-failure compute cost. Feature importance analysis from preliminary experiments confirms that seed sensitivity features contribute the largest share of predictive signal, validating the central hypothesis that seed-pattern analysis is essential for hardware flakiness detection.

C. Feature Ablation Design Rationale

The six feature categories were designed so that each contributes complementary and non-redundant discriminating power. Seed sensitivity and race signatures are expected to provide the largest independent contributions, since they directly address the two most common hardware-specific flaky failure mechanisms. X-propagation and temporal features supply secondary discriminating power for categories that seed analysis alone cannot resolve (e.g., consistent X-induced failures that are nonetheless testbench-originated). Historical pattern features are critical for the cold-start bootstrapping phase but become increasingly important as the failure database matures. Preliminary experiments on small UVM testbenches are consistent with this priority ordering.

D. Case Studies

1) Case 1 — CDC Race in GPU Testbench: A GPU testbench exhibited scoreboard timeout failures in 12% of seeds. Our system identified high CDC activity (87th percentile of training distribution), non-deterministic delay statements in active testbench forks, and timestamp clustering around specific regression positions. Predicted probability: $p(\text{flaky}) = 0.94$; suppressed without rerun. Manual investigation confirmed a race between two testbench clock-crossing processes resolved by adding explicit synchronization.

2) Case 2 — Real Bug Recovered by Adaptive Rerun: A CPU testbench exhibited arithmetic unit failures in approximately 8% of seeds, yielding initial prediction $p(\text{flaky}) = 0.72$, which falls in the borderline zone. Three consecutive reruns all failed, updating the Bayesian posterior to $p(\text{flaky}) = 0.18$, routing the failure to triage. Investigation revealed a corner-case integer overflow triggered by seed values in a specific bit-range—a genuine hardware defect. This case demonstrates that the adaptive rerun policy is essential to correcting initial misclassifications before a real bug is silently suppressed.

V. DISCUSSION

A. False Positive Risk and Escape Monitoring

Suppressing real design bugs is the highest-severity failure mode. We mitigate this through conservative suppression thresholds ($p > 0.8$), mandatory weekly manual review of all suppressed failures, and escape tracking: monitoring whether suppressed failures later reappear as engineer-confirmed bugs. The framework is designed so that even a high-confidence suppression remains auditable and reversible. Weekly reviews are a non-negotiable process control regardless of classifier confidence; they also provide the labeled data needed for ongoing model improvement.

B. Handling Testbench Evolution

Production testbenches evolve continuously as RTL changes propagate. We address this through biweekly model retraining with newly labeled data, transfer learning using the feature extractor and classifier weights when deploying to a new testbench, and distribution-shift detection via confidence calibration monitoring. *Limitation:* Major testbench restructuring events may require more aggressive retraining schedules than the biweekly cadence supports.

C. Training Data Investment

Reaching target precision levels above 90% is expected to require approximately 18 months of labeled historical data in a production regression environment. For new projects without history, the system bootstraps in degraded mode using keyword matching until sufficient labeled data accumulates. *Limitation:* This cold-start period limits immediate applicability to greenfield verification environments. For the open-source validation testbench, controlled failure injection allows label acquisition in compressed time, enabling earlier characterization of framework behavior.

D. Generalization

While developed and validated on UVM, the core framework—seed sensitivity analysis, temporal pattern recognition, adaptive confidence-building—may generalize to other hardware verification methodologies (VMM, OVM) and potentially to software testing with appropriate feature engineering. *Limitation:* We have not validated cross-domain performance; claims of generalization beyond UVM would require separate experimental evaluation.

E. Computational Overhead

Feature extraction adds 2–5 seconds per failure (processing 10–500 MB simulation logs), negligible relative to typical 2–8 hour manual triage sessions and 10–30 minute simulation reruns. For resource-constrained environments in which the 1–5 rerun budget is unavailable, the high-confidence branches ($p > 0.8$ or $p < 0.3$) still operate without any reruns, providing a graceful degradation mode.

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.

VI. Future Work

Four directions are planned. First, **root cause localization**: extending classification to identify specific testbench components (monitors, drivers, scoreboards) responsible for observed flakiness, moving the system from detection to diagnosis. Second, **automated repair**: applying program synthesis techniques [24] to auto-generate fixes for common flaky patterns such as missing fork synchronization or uninitialized variable declarations. Third, **federated cross-project transfer learning**: enabling organizations to share model knowledge across companies while preserving design IP confidentiality. Fourth, **formal methods integration**: combining ML classification with lightweight model checking to provide provable correctness guarantees for race-condition flakiness categories that are amenable to bounded verification.

VII. Conclusion

We presented a machine learning framework for automated flaky test detection and suppression in UVM verification regressions. Our two-stage architecture combines instrumented failure feature extraction, gradient-boosted decision tree classification, and Bayesian adaptive rerun policies. The framework is designed to achieve precision above 90% while averaging fewer than three reruns per failure, with target MTTA reductions of 35–50% relative to manual triage baselines. These targets are grounded in the statistical properties of our structured six-category feature design and analogous results from software flaky test detection research.

The two case studies from industrial regression practice demonstrate the classification logic under realistic conditions: a testbench CDC race correctly suppressed, and a genuine hardware arithmetic defect correctly escalated after adaptive reruns updated the Bayesian posterior. Full quantitative validation on a controlled open-source AHB2APB UVM testbench is in preparation; the testbench infrastructure is in place and measured results will be reported in the extended version.

By automatically suppressing non-actionable flaky failures with high confidence while routing genuine design bugs to immediate engineer attention, the framework removes a critical productivity bottleneck in modern hardware verification. Reduced triage overhead translates directly to faster defect discovery, shorter verification cycles, and lower exposure to schedule delays and silicon respin costs caused by misdirected engineering effort. Future work will extend the system with root-cause diagnosis, automated testbench repair, and cross-project transfer learning to further enhance verification efficiency.

Acknowledgment

The authors thank colleagues in the hardware verification community whose shared experiences with regression flakiness informed the failure taxonomy and case studies presented in this work.

References

[1] J. Bergeron, *Writing Testbenches: Functional Verification of HDL Models*, 2nd ed. Springer Science & Business Media, 2006.

[2] A. Piziali, *Functional Verification Coverage Measurement and Analysis*. Springer, 2013.

[3] H. D. Foster, "Trends in functional verification: A 2014 industry study," in *Proc. 52nd ACM/EDAC/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, Jun. 2015, pp. 1–6.

[4] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proc. 28th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, Beijing, China, Jul. 2019, pp. 101–111.

[5] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *Proc. 40th Int. Conf. Software Engineering (ICSE)*, Gothenburg, Sweden, May 2018, pp. 433–444.

[6] A. Eck, F. Auer, Y. Vogel, and A. Knapp, "Understanding flaky tests: The developer's perspective," in *Proc. 8th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, Ciudad Real, Spain, Sep. 2016, Art. no. 74, 4 pp.

[7] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE)*, Hong Kong, China, Nov. 2014, pp. 643–653.

[8] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in Android apps," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, Madrid, Spain, Sep. 2018, pp. 534–538.

[9] F. Palomba and A. Zaidman, "Does refactoring of test smells induce fixing flaky tests?" in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, Cleveland, OH, USA, Sep. 2019, pp. 1–12.

[10] S. Fine and A. Ziv, "Coverage directed test generation for functional verification using Bayesian networks," in *Proc. 40th Annual Design Automation Conf. (DAC)*, Anaheim, CA, USA, Jun. 2003, pp. 286–291.

[11] J. Yuan, C. Pixley, and A. Aziz, *Constraint-Based Verification*. Springer Science & Business Media, 2006.

[12] E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds., *Handbook of Model Checking*. Springer, 2018.

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.

[13] F. Corno, M. Reorda, and G. Squillero, "Automatic test program generation: A case study," *IEEE Des. & Test Comput.*, vol. 21, no. 2, pp. 102–109, Mar./Apr. 2004.
[14] L. Bening and H. Foster, *Principles of Verifiable RTL Design*, 2nd ed. Springer Science & Business Media, 2001.
[15] R. H. Netzer and B. P. Miller, "What are race conditions? Some issues and formalizations," *ACM Lett. Program. Lang. Syst. (LOPLAS)*, vol. 1, no. 1, pp. 74–88, Mar. 1992.
[16] S. Savage, M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson, "Eraser: A dynamic data race detector for multithreaded programs," *ACM Trans. Comput. Syst.*, vol. 15, no. 4, pp. 391–411, Nov. 1997.
[17] T. Wagner, V. Herdt, S. Schweitzer, and R. Drechsler, "Using machine learning to accelerate constraint solving in coverage-driven verification," in *Proc. Design, Automation & Test in Europe Conf. (DATE)*, Grenoble, France, Mar. 2020, pp. 1415–1420.
[18] T. B. Noor and H. Hemmati, "A similarity-based approach for test case prioritization using historical failure data," in *Proc. IEEE Int. Conf. Software Quality, Reliability and Security (QRS)*, Vancouver, BC, Canada, Aug. 2015, pp. 58–68.
[19] M. J. Harrold, R. Gupta, and M. L. Soffa, "A methodology for controlling the size of a test suite," *ACM Trans. Softw. Eng. Methodol.*, vol. 2, no. 3, pp. 270–285, Jul. 1993.
[20] O. Grinshtein, A. Adir, A. Ziv, and R. Emek, "User defined coverage: A tool supported methodology for design verification," in *Proc. 43rd ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, Jul. 2006, pp. 158–165.
[21] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.
[22] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, Lake Tahoe, NV, USA, Dec. 2012, pp. 2951–2959.
[23] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "FlakeFlagger: Predicting flakiness without rerunning tests," in *Proc. 43rd Int. Conf. Software Engineering (ICSE)*, Madrid, Spain, May 2021, pp. 1572–1584.
[24] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Found. Trends Program. Lang.*, vol. 4, no. 1–2, pp. 1–119, 2017.

Authorized licensed use limited to: National University Fast. Downloaded on August 25,2026 at 09:12:05 UTC from IEEE Xplore. Restrictions apply.